

Notebook 14 - VQC mit optimierten Hyperparametern (Simulator)

Dieses Notebook wiederholt die VQC-Experimente auf dem AerSimulator (ideal und Noise Model) mit den **optimierten Hyperparametern** aus der Hyperparameter-Analyse:

Datensatz	Optimizer	Iterationen	Begründung
Iris	COBYLA	100	Beste Acc/F1 bei geringster Trainingszeit
BC	COBYLA	50	Beste Acc/F1 bei geringster Trainingszeit

Ziel: Direkter Vergleich mit den bisherigen SPSA-150-Ergebnissen sowie Baseline für den Hardware-Lauf.

Imports

```
In [8]: import time
import csv
import os
import numpy as np

from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report

from qiskit.circuit.library import ZZFeatureMap, RealAmplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit.primitives import StatevectorSampler
from qiskit_algorithms.optimizers import COBYLA

from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit_ibm_runtime import SamplerV2 as NoisySampler
from qiskit.transpiler import generate_preset_pass_manager
```

Konfiguration

```
In [12]: OPTIMIZATION_LEVEL = 1    # Pass Manager Level (für Noise-Läufe)
RANDOM_STATE = 42
TEST_SIZE = 0.3
ERGEBNISSE_CSV = 'Ergebnisse/ergebnisse_best_for_hardware_2.csv'

# Optimierte Hyperparameter aus der Analyse
CONFIG = {
```

```
'iris': {'optimizer': COBYLA(maxiter=100), 'maxiter': 100},
'bc':   {'optimizer': COBYLA(maxiter=50), 'maxiter': 50},
}
```

Hilfsfunktionen

```
In [3]: def append_to_csv(row: dict, filepath: str):
        """Hängt eine Ergebniszeile an die CSV-Datei an."""
        fieldnames = ['Modell', 'Datensatz', 'Backend', 'Optimizer', 'Iterationen',
                      'Accuracy', 'F1', 'Trainingszeit_s', 'Inferenzzeit_s']
        file_exists = os.path.isfile(filepath)
        with open(filepath, 'a', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            if not file_exists:
                writer.writeheader()
            writer.writerow(row)
        print(f" → Gespeichert in {filepath}")

def prepare_data(dataset_name: str):
    """Lädt und bereitet einen Datensatz vor (2 Features, fair split)."""
    if dataset_name == 'iris':
        data = load_iris()
        label = 'Iris (3 Klassen, 2 Features, fair)'
        f1_avg = 'weighted'
    else:
        data = load_breast_cancer()
        label = 'Breast Cancer (2 Klassen, 2 Features, fair)'
        f1_avg = 'binary'

    X, y = data.data[:, [0, 2]], data.target
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
    )
    scaler = MinMaxScaler(feature_range=(0, np.pi))
    X_train_sc = scaler.fit_transform(X_train)
    X_test_sc = scaler.transform(X_test)
    return X_train_sc, X_test_sc, y_train, y_test, label, f1_avg, data
```

1. Idealer Simulator - kein Rauschen

1a. Iris - COBYLA, 100 Iterationen

```
In [4]: X_train, X_test, y_train, y_test, ds_label, f1_avg, data = prepare_data('iri

feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
ansatz      = RealAmplitudes(num_qubits=2, reps=2)

vqc = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=100),
```

```

    sampler=StatevectorSampler(),
)

print("Starte VQC Training (Iris, ideal)...")
t0 = time.time()
vqc.fit(X_train, y_train)
train_time = round(time.time() - t0, 4)

t0 = time.time()
y_pred = vqc.predict(X_test)
infer_time = round(time.time() - t0, 4)

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average=f1_avg)

print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
print(f"Training: {train_time}s | Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=data.target_names))

append_to_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA)',
    'Datensatz': ds_label,
    'Backend': 'AerSimulator',
    'Optimizer': 'COBYLA',
    'Iterationen': 100,
    'Accuracy': acc, 'F1': f1,
    'Trainingszeit_s': train_time, 'Inferenzzeit_s': infer_time
}, ERGEBNISSE_CSV)

```

/tmp/ipykernel_1255805/2132450514.py:3: DeprecationWarning: The class ``qiskit.circuit.library.data\_preparation.\_zz\_feature\_map.ZZFeatureMap`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the `zz_feature_map` function as a replacement. Note that this will no longer return a `BlueprintCircuit`, but just a plain `QuantumCircuit`.

```
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
```

/tmp/ipykernel_1255805/2132450514.py:4: DeprecationWarning: The class ``qiskit.circuit.library.n\_local.real\_amplitudes.RealAmplitudes`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the function `qiskit.circuit.library.real_amplitudes` instead.

```
ansatz = RealAmplitudes(num_qubits=2, reps=2)
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Starte VQC Training (Iris, ideal)...

Accuracy: 0.6222 | F1: 0.4974

Training: 50.6673s | Inferenz: 0.3482s

	precision	recall	f1-score	support
setosa	0.65	1.00	0.79	15
versicolor	0.59	0.87	0.70	15
virginica	0.00	0.00	0.00	15
accuracy			0.62	45
macro avg	0.41	0.62	0.50	45
weighted avg	0.41	0.62	0.50	45

→ Gespeichert in ergebnisse.csv

```

/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

1b. Breast Cancer - COBYLA, 50 Iterationen

```

In [5]: X_train, X_test, y_train, y_test, ds_label, f1_avg, data = prepare_data('bc')

feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
ansatz      = RealAmplitudes(num_qubits=2, reps=2)

vqc = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=50),
    sampler=StatevectorSampler(),
)

print("Starte VQC Training (BC, ideal)...")
t0 = time.time()
vqc.fit(X_train, y_train)
train_time = round(time.time() - t0, 4)

t0 = time.time()
y_pred = vqc.predict(X_test)
infer_time = round(time.time() - t0, 4)

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average=f1_avg)

print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
print(f"Training: {train_time}s | Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=data.target_names))

append_to_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA)',
    'Datensatz': ds_label,
    'Backend': 'AerSimulator',
    'Optimizer': 'COBYLA',
    'Iterationen': 50,
    'Accuracy': acc, 'F1': f1,
})

```

```
'Trainingszeit_s': train_time, 'Inferenzzeit_s': infer_time
}, ERGEBNISSE_CSV)
```

```
/tmp/ipykernel_1255805/2167248219.py:3: DeprecationWarning: The class ``qiskit.circuit.library.data_preparation._zz_feature_map.ZZFeatureMap`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the zz_feature_map function as a replacement. Note that this will no longer return a BlueprintCircuit, but just a plain QuantumCircuit.
```

```
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
/tmp/ipykernel_1255805/2167248219.py:4: DeprecationWarning: The class ``qiskit.circuit.library.n_local.real_amplitudes.RealAmplitudes`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the function qiskit.circuit.library.real_amplitudes instead.
```

```
ansatz = RealAmplitudes(num_qubits=2, reps=2)
No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.
```

Starte VQC Training (BC, ideal)...

Accuracy: 0.8363 | F1: 0.8727

Training: 151.3932s | Inferenz: 1.2656s

	precision	recall	f1-score	support
malignant	0.81	0.73	0.77	64
benign	0.85	0.90	0.87	107
accuracy			0.84	171
macro avg	0.83	0.82	0.82	171
weighted avg	0.83	0.84	0.83	171

→ Gespeichert in ergebnisse.csv

2. Noise Model Simulator

```
In [6]: # Noise Model (identisch zu bisherigen Experimenten)
noise_model = NoiseModel()
error_1q = depolarizing_error(0.001, 1)
error_2q = depolarizing_error(0.01, 2)
noise_model.add_all_qubit_quantum_error(error_1q, ['h', 'x', 'u1', 'u2', 'u3'])
noise_model.add_all_qubit_quantum_error(error_2q, ['cx'])

noisy_backend = AerSimulator(noise_model=noise_model)
pm = generate_preset_pass_manager(
    optimization_level=OPTIMIZATION_LEVEL,
    backend=noisy_backend
)
print(f"Noise Model konfiguriert | Pass Manager Level: {OPTIMIZATION_LEVEL}")
```

Noise Model konfiguriert | Pass Manager Level: 1

2a. Iris - COBYLA, 100 Iterationen, Noise

```
In [8]: X_train, X_test, y_train, y_test, ds_label, f1_avg, data = prepare_data('iris')

feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
ansatz = RealAmplitudes(num_qubits=2, reps=2)
```

```

sampler_noisy = NoisySampler(noisy_backend)

vqc_noise = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=100),
    sampler=sampler_noisy,
    pass_manager=pm,
)

print("Starte VQC Training (Iris, Noise)...")
t0 = time.time()
vqc_noise.fit(X_train, y_train)
train_time = round(time.time() - t0, 4)

t0 = time.time()
y_pred = vqc_noise.predict(X_test)
infer_time = round(time.time() - t0, 4)

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average=f1_avg)

print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
print(f"Training: {train_time}s | Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=data.target_names))

append_to_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA, Noise)',
    'Datensatz': ds_label.replace('fair', 'noise'),
    'Backend': 'AerSimulator+Noise',
    'Optimizer': 'COBYLA',
    'Iterationen': 100,
    'Accuracy': acc, 'F1': f1,
    'Trainingszeit_s': train_time, 'Inferenzzeit_s': infer_time
}, ERGEBNISSE_CSV)

```

/tmp/ipykernel_1255805/3801721246.py:3: DeprecationWarning: The class ``qiskit.circuit.library.data\_preparation.\_zz\_feature\_map.ZZFeatureMap`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the `zz_feature_map` function as a replacement. Note that this will no longer return a `BlueprintCircuit`, but just a plain `QuantumCircuit`.

```
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
```

/tmp/ipykernel_1255805/3801721246.py:4: DeprecationWarning: The class ``qiskit.circuit.library.n\_local.real\_amplitudes.RealAmplitudes`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the function `qiskit.circuit.library.real_amplitudes` instead.

```
ansatz = RealAmplitudes(num_qubits=2, reps=2)
```

Starte VQC Training (Iris, Noise)...

Accuracy: 0.6222 | F1: 0.4972

Training: 56.7252s | Inferenz: 0.4226s

	precision	recall	f1-score	support
setosa	0.62	1.00	0.77	15
versicolor	0.62	0.87	0.72	15
virginica	0.00	0.00	0.00	15
accuracy			0.62	45
macro avg	0.41	0.62	0.50	45
weighted avg	0.41	0.62	0.50	45

→ Gespeichert in ergebnisse.csv

```
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/home/fabian/anaconda3/envs/qml/lib/python3.11/site-packages/sklearn/metric
s/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined
and being set to 0.0 in labels with no predicted samples. Use `zero_division
` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

2b. Breast Cancer - COBYLA, 50 Iterationen, Noise

```
In [9]: X_train, X_test, y_train, y_test, ds_label, f1_avg, data = prepare_data('bc')

feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
ansatz      = RealAmplitudes(num_qubits=2, reps=2)

sampler_noisy = NoisySampler(noisy_backend)

vqc_noise = VQC(
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=COBYLA(maxiter=50),
    sampler=sampler_noisy,
    pass_manager=pm,
)

print("Starte VQC Training (BC, Noise)...")
t0 = time.time()
vqc_noise.fit(X_train, y_train)
train_time = round(time.time() - t0, 4)

t0 = time.time()
y_pred = vqc_noise.predict(X_test)
infer_time = round(time.time() - t0, 4)
```

```

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average=f1_avg)

print(f"Accuracy: {acc:.4f} | F1: {f1:.4f}")
print(f"Training: {train_time}s | Inferenz: {infer_time}s")
print(classification_report(y_test, y_pred, target_names=data.target_names))

append_to_csv({
    'Modell': 'VQC (RealAmplitudes, COBYLA, Noise)',
    'Datensatz': ds_label.replace('fair', 'noise'),
    'Backend': 'AerSimulator+Noise',
    'Optimizer': 'COBYLA',
    'Iterationen': 50,
    'Accuracy': acc, 'F1': f1,
    'Trainingszeit_s': train_time, 'Inferenzzeit_s': infer_time
}, ERGEBNISSE_CSV)

```

Starte VQC Training (BC, Noise)...

/tmp/ipykernel_1255805/2868223600.py:3: DeprecationWarning: The class ``qiskit.circuit.library.data\_preparation.zz\_feature\_map.ZZFeatureMap`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the zz_feature_map function as a replacement. Note that this will no longer return a BlueprintCircuit, but just a plain QuantumCircuit.

```
feature_map = ZZFeatureMap(feature_dimension=2, reps=1)
```

/tmp/ipykernel_1255805/2868223600.py:4: DeprecationWarning: The class ``qiskit.circuit.library.n\_local.real\_amplitudes.RealAmplitudes`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. Use the function qiskit.circuit.library.real_amplitudes instead.

```
ansatz = RealAmplitudes(num_qubits=2, reps=2)
```

Accuracy: 0.8246 | F1: 0.8661

Training: 139.7184s | Inferenz: 0.9717s

	precision	recall	f1-score	support
malignant	0.81	0.69	0.75	64
benign	0.83	0.91	0.87	107
accuracy			0.82	171
macro avg	0.82	0.80	0.81	171
weighted avg	0.82	0.82	0.82	171

→ Gespeichert in ergebnisse.csv

Zusammenfassung

In [14]: `import pandas as pd`

```

df = pd.read_csv(ERGEBNISSE_CSV)
print(df.to_string(index=False))

```

Modell		Datensatz				
Backend	Optimizer	Iterationen	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, fair)			AerS
imulator	COBYLA	100	0.6222	0.4973	50.6673	
						0.3482
VQC (RealAmplitudes)			Breast Cancer (2 Klassen, 2 Features, fair)			AerS
imulator	COBYLA	50	0.8362	0.8727	151.3932	
						1.2656
VQC (RealAmplitudes)			Iris (3 Klassen, 2 Features, noise)			AerSimulat
or+Noise	COBYLA	100	0.6222	0.4971	56.7252	
						0.4226
VQC (RealAmplitudes)			Breast Cancer (2 Klassen, 2 Features, noise)			AerSimulat
or+Noise	COBYLA	50	0.8245	0.8660	139.7184	
						0.9717